

Proyecto: Cloud Provider Analytics

62466 - Juan Ignacio Fernández Dinardo

60401 - Martín E. Zahnd

Contexto

Como equipo de datos de un proveedor de nube tenemos la misión de ingestar, limpiar, conformar y publicar datos de clientes para analítica de FinOps, Soporte y Producto. Se nos exigen dos capacidades:

- Near real-time para métricas operativas (uso/consumo, costos incrementales).
- Batch diario/mensual para maestros (CRM) y facturación.

Los datos aterrizan crudos con inconsistencias, nulos, tipos ambiguos, anomalías y una evolución de esquema a mitad del histórico (aparecen campos nuevos como `carbon_kg` y `genai_tokens`).

Nuestro objetivo es construir un pipeline con PySpark en Google Colab, usando Parquet como storage intermedio gestionado por Spark, y publicar marts de analítica en AstraDB (Cassandra) para ser consultados por herramientas de visualización (Tableau/Power BI/Superset/Grafana, etc).

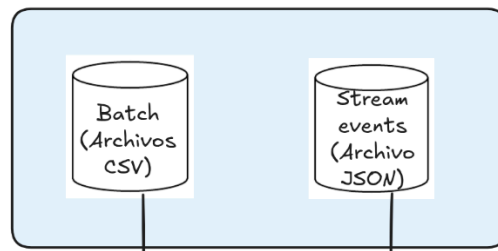
Diagrama de Arquitectura de Alto Nivel

Dada la naturaleza híbrida de los datos, el proyecto implementa una arquitectura Lambda que combina dos flujos de procesamiento diferenciados según la naturaleza de los datos. Esto permite aprovechar los recursos limitados de Colab, optimizando para cada tipo de datos, y evita el over-engineering al forzar todo por streaming si se quisiera utilizar el patrón Kappa, pues implicaría un gran costo de desarrollo y recursos para información que no precisa ser procesada y visualizada “en el instante”.

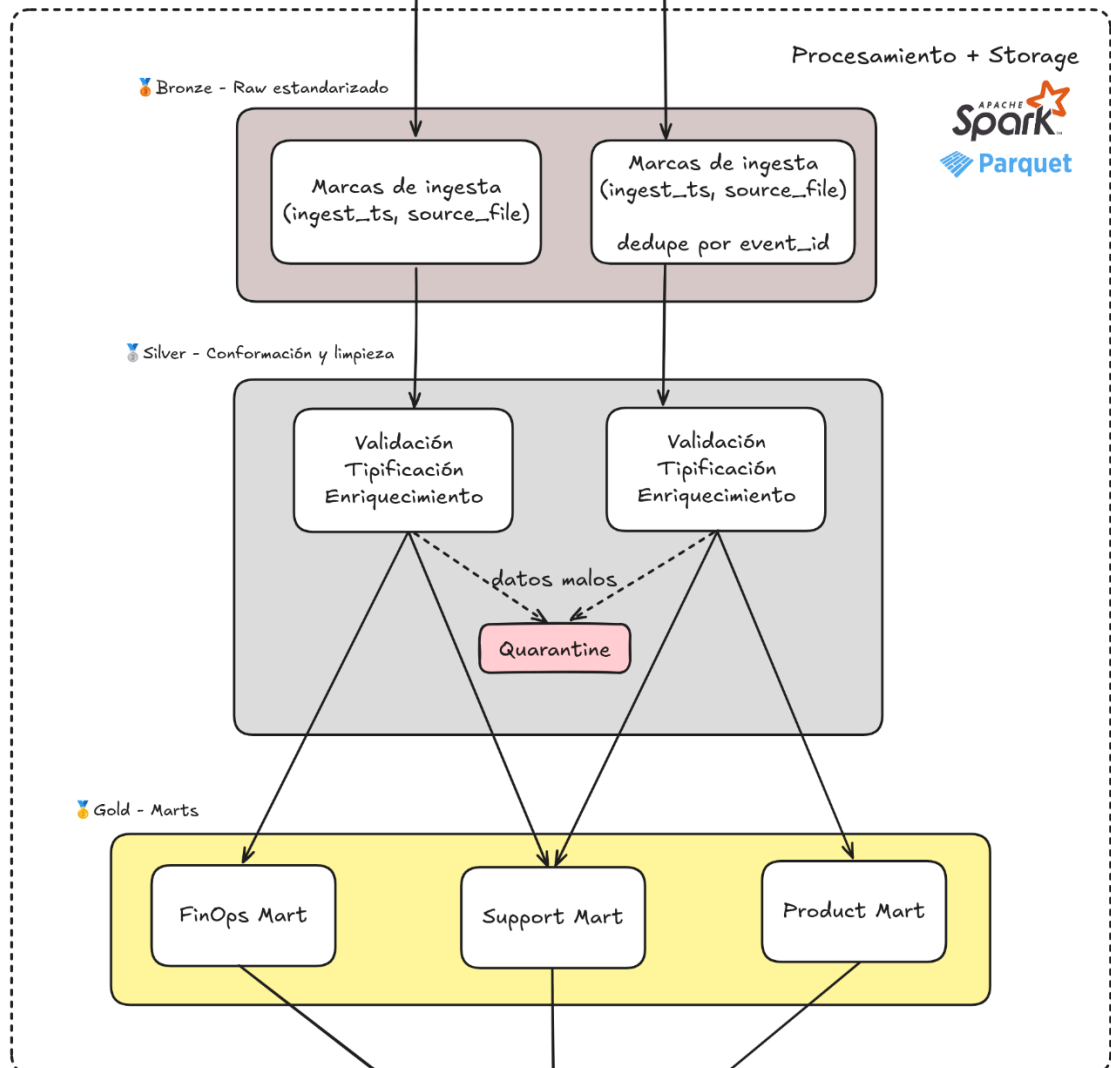
El path batch procesa maestros como customers, users, resources, así como facturación y tickets de soporte, los cuales tienen una naturaleza estática o se actualizan diariamente, por lo que el procesamiento batch resulta suficiente y eficiente. Por otro lado, el path streaming maneja los eventos de uso (`usage_events_stream`) mediante Spark Structured Streaming con micro-lotes, garantizando la latencia baja requerida para métricas operativas en tiempo cercano al real.

Ambos paths convergen en la capa Gold, donde alimentan los mismos marts de negocio antes de ser servidos a través de AstraDB (Cassandra), permitiendo así una visión unificada de los datos históricos y en tiempo real para análisis de FinOps, Soporte y Producto.

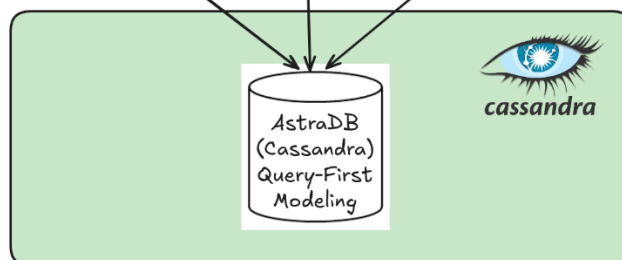
Data Source



Data Lake



Serving layer



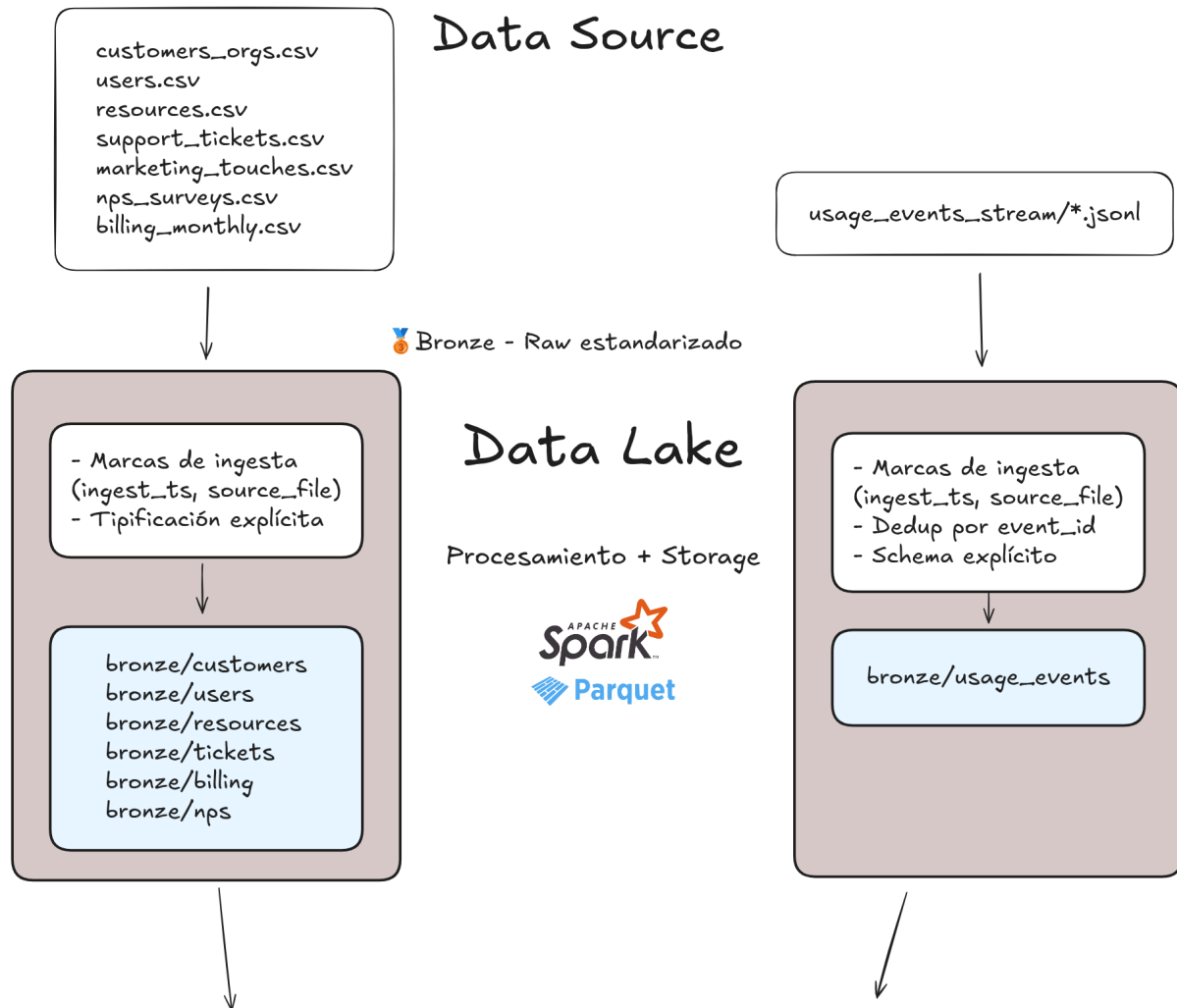
Mapeo de Requisitos a Componentes

Requisito	Componente/Tecnología	Justificación
Near real-time para métricas operativas	Spark Structured Streaming	Procesamiento incremental con latencia <1min, manejo de late data
Batch diario/mensual para maestros	Spark Batch (read.csv) + particionado	Eficiente para datos estáticos, permite overwrite por partición
Almacenamiento escalable	Parquet particionado (date, service)	Compresión 10-100x, columnar, predicate pushdown
Calidad de datos	Validaciones + Quarantine zone	Separa datos inválidos sin detener pipeline
Evolución de esquema	mergeSchema en Silver + compatibilización	Unifica v1 (sin carbon_kg) y v2 (con carbon_kg, genai_tokens)
Idempotencia	Checkpointing (streaming) + keys naturales	Reprocesamiento sin duplicados
Serving analítico	AstraDB (Cassandra)	Modelado query-first, escalabilidad horizontal, baja latencia

V	Descripción en el Contexto	Abordaje
Volumen	~500K eventos/día + maestros (estimado <100GB/día)	Parquet comprimido + particionado
Velocidad	Eventos streaming (segundos), batch (diario)	Dual-path: Structured Streaming + Batch jobs
Variedad	CSV, JSON, JSONL; estructurados + semi-estructurados	Schema explícito por tipo, conformance en Silver
Veracidad	Nulos, duplicados, anomalías, inconsistencias	Validaciones, quarantine, detección de outliers
Valor	Métricas FinOps, SLA support, insights GenAI	Marts especializados Gold, KPIs pre-calculados

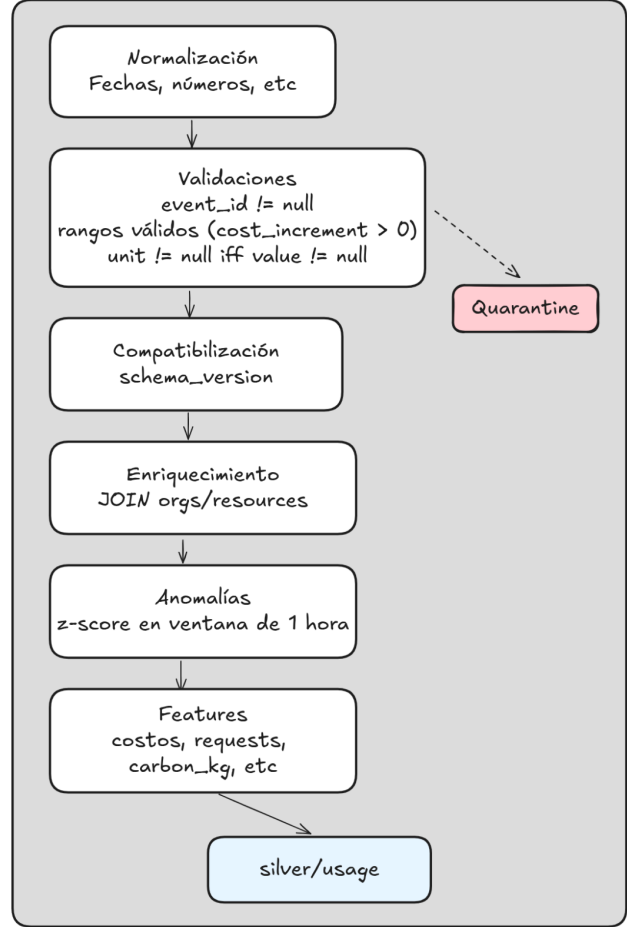
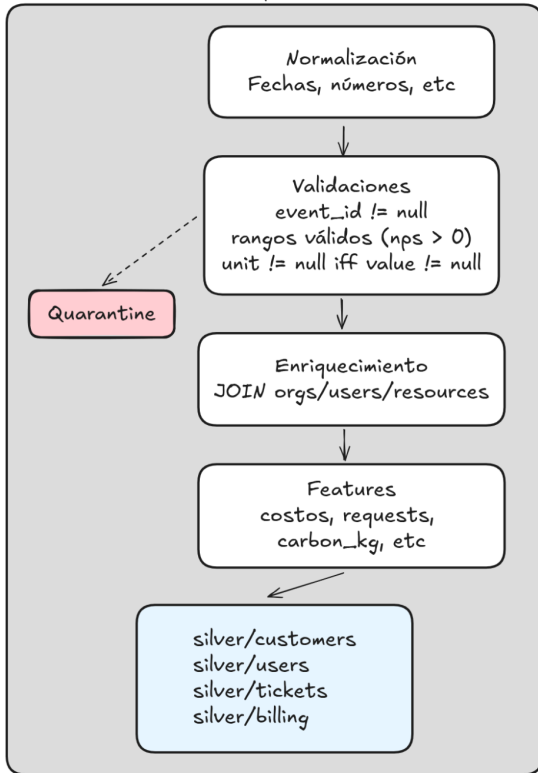
Flujo de Datos (Data Pipeline)

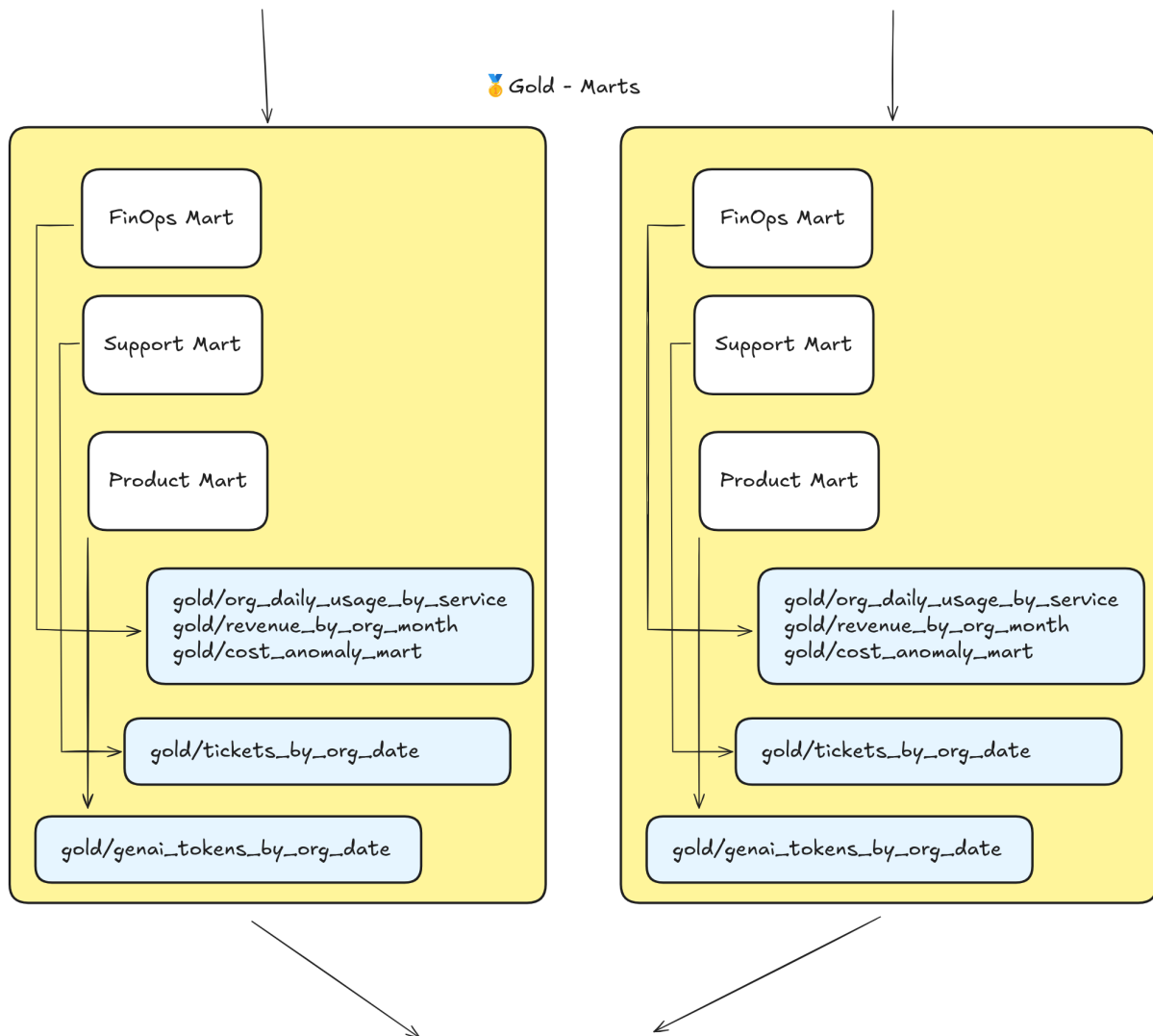
Para facilitar la lectura de esta sección, decidimos dividir el gráfico en varios sub-gráficos contiguos, presentando distintas capas del Data Pipeline.



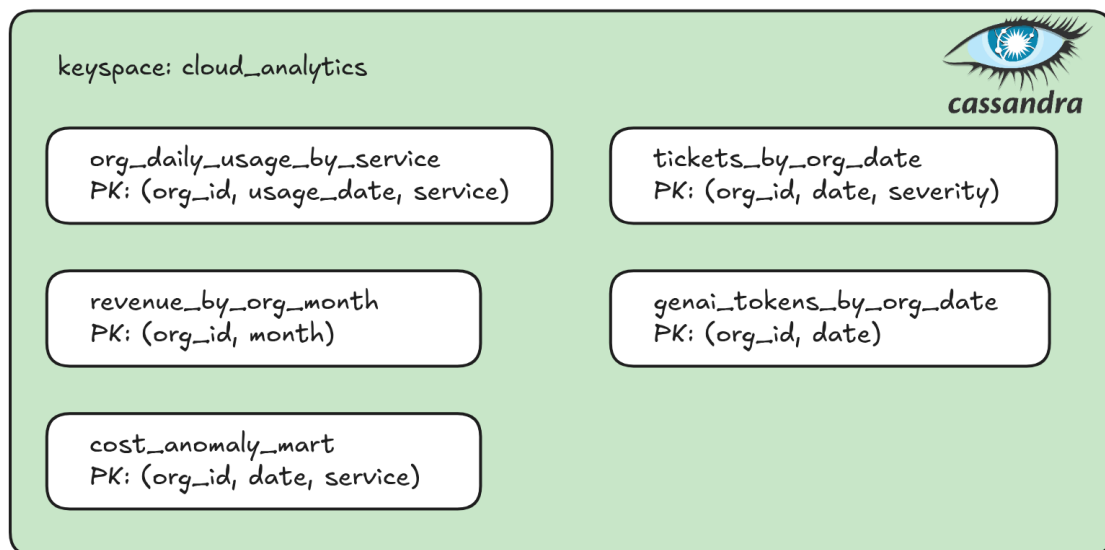
Nótese que, a la izquierda, se encuentra el procesamiento batch, y a la derecha el streaming. En ambos casos, en el recuadro azul se diagrama el output que se almacena en Parquet. Esto se repite en Silver y Gold

Silver - Conformación y limpieza





Serving layer



Asunciones y Riesgos Iniciales

Asunciones:

1. **Volumen inicial <10GB/día:** Colab (15 GB de RAM) es suficiente con particionado adecuado para procesar los datos sin necesidad de infraestructura más robusta.
2. **Eventos llegan ordenados por timestamp en ventanas de 1h:** Se asume que el desorden temporal de los eventos no supera una hora, por lo que un watermark de 1h es suficiente para cubrir late data.
3. **Schema v2 aparece a partir del día 45 y coexiste con v1:** La evolución de esquema se maneja mediante `mergeSchema=true` en Spark junto con compatibilización manual de campos como `carbon_kg` y `genai_tokens`.
4. **AstraDB free tier (80GB storage) es suficiente para marts Gold:** Se asume una retención de 90 días en Gold, lo cual entra dentro de los límites del tier gratuito de AstraDB.
5. **Sin broker dedicado (Kafka); simula streaming con archivos:** La simulación de streaming mediante archivos JSONL proporciona una latencia menor a 2 minutos, aceptable para propósitos demostrativos.

Riesgos:

1. **OOM (Out of Memory) en Colab con datasets grandes**
[Probabilidad: *Media* | Impacto: *Alto*]
 - *Mitigación:* Procesamiento incremental por particiones, uso de `coalesce()` para reducir paralelismo, y considerar Colab Pro si es necesario.
2. **Late data >1h no capturado**
[Probabilidad: *Baja* | Impacto: *Medio*]
 - *Mitigación:* Ajustar watermark a 2-3 horas según comportamiento observado, monitorear métricas de eventos descartados (dropped events).
3. **Schema drift no detectado a tiempo**
[Probabilidad: *Media* | Impacto: *Alto*]
 - *Mitigación:* Implementar tests automáticos de schema en Bronze, configurar alerts en logs cuando aparezcan campos inesperados.
4. **Duplicados en Cassandra si falla idempotencia**
[Probabilidad: *Media* | Impacto: *Alto*]
 - *Mitigación:* Usar operaciones UPSERT con Primary Key correctamente definida, mantener checkpoints estrictos en Structured Streaming.
5. **Performance degradado en joins de Silver (broadcast)**
[Probabilidad: *Baja* | Impacto: *Medio*]

- *Mitigación:* Mantener dimensiones pequeñas (<10MB) en caché, utilizar broadcast hints explícitos para forzar broadcast joins.

Estimación de Esfuerzo y Recursos

Asumiendo un equipo de dos personas, se estima que el proyecto tendrá las siguientes fases y estimaciones:

1. Diseño y Planificación: 2 días
2. Setup de la infraestructura: 1 a 3 días
3. Ingest y Bronze Layer: 3 días
4. Silver Layer: 5 a 7 días
5. Marts Gold: 3 a 5 días
6. Integración con Cassandra: 2 a 3 días
7. Testing y validación: 5 días

Total (máximo): 33 días hábiles (aproximadamente 7 semanas laborales)

Un integrante del equipo tendrá la responsabilidad del setup de la infraestructura (configuración de Colab, instalación de dependencias, configuración de Spark), y las distintas layers de modo batch; mientras el otro integrante será responsable de las layers del modo streaming, y el modelado e integración con Cassandra.

Ambos realizarán el diseño y planificación en conjunto, al igual que la fase de testing y validación.

En cuanto a recursos, utilizaremos el free tier de Google Colab (15GB de RAM, 12 horas de sesión); el free tier de AstraDB (80 GB de storage, 20M ops/mes); y Google Drive (15 GB) para el almacenamiento de datasets y Parquet intermedio.

Si se considera necesario actualizar a Google Colab Pro, el mismo cuesta USD 10 al mes. Si el volumen de datos alcanza los 10 GB por día, Google Cloud Storage tiene un costo de USD 0,020 por GB por mes de almacenamiento¹, lo que implica un costo de USD 0,20 por mes extra.

Por lo tanto, el proyecto será gratuito (USD 0), o tendrá un costo aproximado de USD 10,20 mensuales, dependiendo del volumen de datos.

¹ Ver más detalles en: <https://cloud.google.com/storage/pricing>